CP ALGORITHM TEMPLATE + WHEN TO USE

Purpose:

Keep this file as a quick CP revision/reference sheet.

For each algorithm/data structure:

1. What it does

2. When to use it

3. Complexity

4. Copy-paste template

0. MASTER CP TEMPLATE

WHEN TO USE:

Use as the base for almost every Codeforces/competitive programming problem.

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using ld = long double;

#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()

const ll INF = 4e18;
const ll MOD = 1e9 + 7;

ll binpow(ll a, ll b, ll mod = MOD) {
    ll res = 1;
    while(b) {
        if(b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

void solve() {

}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T = 1;
    cin >> T;

    while(T--)
        solve();

    return 0;
}
```

1. PREFIX SUM

WHEN TO USE:

• Many range-sum queries.

• Need sum of [l, r] quickly.

• Problems involving subarray sums.

COMPLEXITY:

Preprocessing O(n), query O(1).

```
vector<ll> pref(n + 1, 0);

for(int i = 0; i < n; i++)
    pref[i + 1] = pref[i] + a[i];

ll sum = pref[r + 1] - pref[l];
```

2. SUFFIX SUM

WHEN TO USE:

• Need sum from i to n-1.

• Split array into left/right portions.

• Combine prefix answer and suffix answer.

COMPLEXITY:

O(n) preprocessing, O(1) query.

```
vector<ll> suff(n + 1, 0);

for(int i = n - 1; i >= 0; i--)
    suff[i] = suff[i + 1] + a[i];

ll sum = suff[l] - suff[r + 1];
```

3. PREFIX / SUFFIX MINIMUM

WHEN TO USE:

• Need minimum on the left/right of every index.

• Split-array problems.

• Greedy problems.

```
vector<ll> prefMin(n), suffMin(n);

prefMin[0] = a[0];
for(int i = 1; i < n; i++)
    prefMin[i] = min(prefMin[i - 1], a[i]);

suffMin[n - 1] = a[n - 1];
for(int i = n - 2; i >= 0; i--)
    suffMin[i] = min(suffMin[i + 1], a[i]);
```

4. PREFIX / SUFFIX MAXIMUM

WHEN TO USE:

• Need maximum on either side of every index.

• Split-array problems.

• Stock/building/greedy problems.

```
vector<ll> prefMax(n), suffMax(n);

prefMax[0] = a[0];
for(int i = 1; i < n; i++)
    prefMax[i] = max(prefMax[i - 1], a[i]);

suffMax[n - 1] = a[n - 1];
for(int i = n - 2; i >= 0; i--)
    suffMax[i] = max(suffMax[i + 1], a[i]);
```

5. PREFIX / SUFFIX GCD

WHEN TO USE:

• GCD of array after removing one element.

• Need GCD of left and right parts.

• Number theory + arrays.

```
vector<ll> prefGcd(n + 1, 0), suffGcd(n + 1, 0);

for(int i = 0; i < n; i++)
    prefGcd[i + 1] = gcd(prefGcd[i], a[i]);

for(int i = n - 1; i >= 0; i--)
    suffGcd[i] = gcd(suffGcd[i + 1], a[i]);

// GCD after removing a[i]
ll g = gcd(prefGcd[i], suffGcd[i + 1]);
```

6. PREFIX / SUFFIX XOR

WHEN TO USE:

• Range XOR.

• XOR subarray problems.

• Prefix XOR + bitwise problems.

```
vector<ll> prefXor(n + 1, 0);

for(int i = 0; i < n; i++)
    prefXor[i + 1] = prefXor[i] ^ a[i];

ll x = prefXor[r + 1] ^ prefXor[l];
```

Suffix:

```
vector<ll> suffXor(n + 1, 0);

for(int i = n - 1; i >= 0; i--)
    suffXor[i] = suffXor[i + 1] ^ a[i];
```

7. DIFFERENCE ARRAY

WHEN TO USE:

• Many range additions.

• Add x to every element in [l, r].

• Need final array after all range updates.

COMPLEXITY:

O(1) per update, O(n) reconstruction.

```
vector<ll> diff(n + 1, 0);

diff[l] += val;
diff[r + 1] -= val;

for(int i = 1; i < n; i++)
    diff[i] += diff[i - 1];
```

8. 2D PREFIX SUM

WHEN TO USE:

• Grid rectangle sum queries.

• Matrix/submatrix problems.

```
vector<vector<ll>> pref(n + 1, vector<ll>(m + 1, 0));

for(int i = 1; i <= n; i++) {
    for(int j = 1; j <= m; j++) {
        pref[i][j] = a[i-1][j-1]
                   + pref[i-1][j]
                   + pref[i][j-1]
                   - pref[i-1][j-1];
    }
}

// rectangle (x1,y1) to (x2,y2), 1-indexed
ll sum = pref[x2][y2]
```

```
   - pref[x1-1][y2]
   - pref[x2][y1-1]
   + pref[x1-1][y1-1];
```

9. SLIDING WINDOW - FIXED SIZE

WHEN TO USE:

• Every window has exactly k elements.

• Maximum/minimum/sum of subarrays of size k.

```
ll sum = 0;

for(int i = 0; i < n; i++) {
    sum += a[i];

    if(i >= k)
        sum -= a[i-k];

    if(i >= k-1) {
        // process window [i-k+1, i]
    }
}
```

10. SLIDING WINDOW - VARIABLE SIZE

WHEN TO USE:

• Longest/shortest subarray satisfying a condition.

• Especially useful when expanding right and shrinking left maintains a monotonic condition.

• Common with positive numbers, frequency constraints, distinct elements.

```
int l = 0;

for(int r = 0; r < n; r++) {

    // add a[r]

    while(/* window invalid */) {
        // remove a[l]
        l++;
    }

    // [l, r] is valid
}
```

11. TWO POINTERS

WHEN TO USE:

• Sorted array.

• Pair/triplet problems.

• Find pair with target sum.

• Merge-like scanning of two arrays.

• Often replaces O(n^2) with O(n).

```
int l = 0, r = n - 1;

while(l < r) {
    ll sum = a[l] + a[r];

    if(sum == target) {
        // found
        l++;
        r--;
    }
    else if(sum < target)
        l++;
    else
        r--;
}
```

12. KADANE'S ALGORITHM

WHEN TO USE:

• Maximum subarray sum.

• Maximum sum of a contiguous subarray.

COMPLEXITY:

O(n).

```
ll cur = 0, ans = LLONG_MIN;

for(ll x : a) {
    cur = max(x, cur + x);
    ans = max(ans, cur);
}
```

13. STACK

WHEN TO USE:

• Matching brackets.

• Undo/backtracking-like behavior.

• Expression parsing.

• Problems where the latest unresolved item must be processed first.

```
stack<int> st;

st.push(x);
st.top();
st.pop();
st.empty();
st.size();
```

14. QUEUE

WHEN TO USE:

• BFS.

• Process elements in insertion order.

• Level-order traversal.

```
queue<int> q;

q.push(x);
q.front();
q.pop();
q.empty();
```

15. DEQUE

WHEN TO USE:

• Need insertion/deletion from both ends.

• Sliding window.

• Monotonic queue.

```
deque<int> dq;

dq.push_front(x);
dq.push_back(x);

dq.pop_front();
dq.pop_back();

dq.front();
dq.back();
```

16. MONOTONIC STACK - NEXT GREATER

WHEN TO USE:

• Next greater element.

• First greater element to left/right.

• Histogram problems.

• Stock span.

• Contribution problems.

```
vector<int> nge(n, -1);
stack<int> st;

for(int i = n - 1; i >= 0; i--) {

    while(!st.empty() && a[st.top()] <= a[i])
        st.pop();

    if(!st.empty())
        nge[i] = a[st.top()];

    st.push(i);
}
```

17. MONOTONIC STACK - NEXT SMALLER

WHEN TO USE:

• Next smaller element.

• Histogram.

• Minimum contribution problems.

```
vector<int> nse(n, -1);
stack<int> st;

for(int i = n - 1; i >= 0; i--) {

    while(!st.empty() && a[st.top()] >= a[i])
        st.pop();

    if(!st.empty())
        nse[i] = a[st.top()];

    st.push(i);
}
```

18. MONOTONIC QUEUE - SLIDING WINDOW MAX

WHEN TO USE:

• Maximum/minimum of every window of size k.

• Need O(n) instead of O(nk).

```
deque<int> dq;

for(int i = 0; i < n; i++) {

    while(!dq.empty() && dq.front() <= i-k)
        dq.pop_front();

    while(!dq.empty() && a[dq.back()] <= a[i])
        dq.pop_back();

    dq.push_back(i);

    if(i >= k-1)
        cout << a[dq.front()] << '\n';
}
```

For minimum, use >= instead of <=.

19. LINKED LIST - REVERSE

WHEN TO USE:

• Reverse singly linked list.

• Reversing a portion of a list.

• Foundation for many linked-list problems.

```
ListNode* reverseList(ListNode* head) {

    ListNode* prev = nullptr;
    ListNode* cur = head;

    while(cur) {
        ListNode* nxt = cur->next;
        cur->next = prev;
        prev = cur;
        cur = nxt;
    }

    return prev;
}
```

20. LINKED LIST - FAST AND SLOW POINTER

WHEN TO USE:

• Find middle.

• Detect cycle.

• Find cycle entry.

• Palindrome linked list.

Find middle:

```
ListNode *slow = head;
ListNode *fast = head;

while(fast && fast->next) {
    slow = slow->next;
    fast = fast->next->next;
}
```

Cycle detection:

```
ListNode *slow = head;
ListNode *fast = head;

while(fast && fast->next) {
    slow = slow->next;
    fast = fast->next->next;

    if(slow == fast)
        return true;
}

return false;
```

21. STRING FREQUENCY

WHEN TO USE:

• Character counting.

• Anagrams.

• Frequency-based string problems.

```
vector<int> freq(26, 0);

for(char c : s)
    freq[c - 'a']++;
```

22. PREFIX / SUFFIX STRING FREQUENCY

WHEN TO USE:

• Count characters in prefixes/suffixes.

• Split a string at every position.

• Need frequency comparison across a split.

```
vector<array<int,26>> pref(n + 1), suff(n + 1);

for(int i = 0; i < n; i++) {
    pref[i+1] = pref[i];
    pref[i+1][s[i] - 'a']++;
}

for(int i = n - 1; i >= 0; i--) {
    suff[i] = suff[i+1];
    suff[i][s[i] - 'a']++;
}
```

23. KMP / PREFIX FUNCTION

WHEN TO USE:

• Pattern matching.

• Find occurrences of pattern in text.

• Borders/prefix-suffix matching.

• String periodicity.

COMPLEXITY:

O(n).

```
vector<int> prefix_function(string s) {

    int n = s.size();
    vector<int> pi(n);

    for(int i = 1; i < n; i++) {

        int j = pi[i-1];

        while(j > 0 && s[i] != s[j])
            j = pi[j-1];

        if(s[i] == s[j])
            j++;

        pi[i] = j;
    }

    return pi;
}
```

Pattern search:

```
string t = pattern + "#" + text;
vector<int> pi = prefix_function(t);

for(int i = 0; i < (int)t.size(); i++) {
    if(pi[i] == (int)pattern.size()) {
        // occurrence ends at i
    }
}
```

24. Z ALGORITHM

WHEN TO USE:

• Pattern matching.

• For every position, need length of longest prefix match.

• Prefix-related string problems.

• Alternative to KMP.

```
vector<int> z_function(string s) {
```

```
    int n = s.size();
    vector<int> z(n);

    for(int i = 1, l = 0, r = 0; i < n; i++) {

        if(i <= r)
            z[i] = min(r-i+1, z[i-l]);

        while(i + z[i] < n &&
              s[z[i]] == s[i+z[i]])
            z[i]++;

        if(i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }

    return z;
}
```

25. MANACHER'S ALGORITHM

WHEN TO USE:

• Longest palindromic substring.

• Count/find palindromes efficiently.

• Need palindrome radius at every position.

COMPLEXITY:

O(n).

```
vector<int> manacher_odd(string s) {

    int n = s.size();
    vector<int> d1(n);

    for(int i = 0, l = 0, r = -1; i < n; i++) {

        int k = (i > r) ? 1 : min(d1[l+r-i], r-i+1);

        while(i-k >= 0 && i+k < n && s[i-k] == s[i+k])
            k++;

        d1[i] = k--;

        if(i+k > r) {
            l = i-k;
            r = i+k;
        }
    }

    return d1;
}
```

This version handles odd-length palindromes. Keep an even-length version in your advanced string notes if needed.

26. TRIE

WHEN TO USE:

• Prefix queries.

• Dictionary of strings.

• Autocomplete-like problems.

• Maximum/minimum word prefix.

• XOR trie variant for integers.

```
struct Trie {

    struct Node {
        int child[26];
        bool end;

        Node() {
            fill(child, child + 26, -1);
            end = false;
```

```
        }
    };

    vector<Node> tr;

    Trie() {
        tr.emplace_back();
    }

    void insert(string s) {

        int u = 0;

        for(char c : s) {
            int x = c - 'a';

            if(tr[u].child[x] == -1) {
                tr[u].child[x] = tr.size();
                tr.emplace_back();
            }

            u = tr[u].child[x];
        }

        tr[u].end = true;
    }

    bool search(string s) {

        int u = 0;

        for(char c : s) {
            int x = c - 'a';

            if(tr[u].child[x] == -1)
                return false;

            u = tr[u].child[x];
        }

        return tr[u].end;
    }
};
```

27. BINARY TRIE

WHEN TO USE:

• Maximum XOR pair.

• Minimum XOR.

• XOR queries.

• Bitwise optimization.

```
struct BinaryTrie {

    struct Node {
        int child[2];

        Node() {
            child[0] = child[1] = -1;
        }
    };

    vector<Node> tr;

    BinaryTrie() {
        tr.emplace_back();
    }

    void insert(int x) {

        int u = 0;

        for(int b = 30; b >= 0; b--) {

            int bit = (x >> b) & 1;

            if(tr[u].child[bit] == -1) {
                tr[u].child[bit] = tr.size();
                tr.emplace_back();
            }

            u = tr[u].child[bit];
        }
    }
```

```
    int maxXor(int x) {

        int u = 0;
        int ans = 0;

        for(int b = 30; b >= 0; b--) {

            int bit = (x >> b) & 1;
            int want = bit ^ 1;

            if(tr[u].child[want] != -1) {
                ans |= (1 << b);
                u = tr[u].child[want];
            }
            else {
                u = tr[u].child[bit];
            }
        }

        return ans;
    }
};
```

28. BINARY SEARCH - NORMAL

WHEN TO USE:

• Sorted array.

• Find element/position.

• Monotonic predicate.

```
int l = 0, r = n - 1;

while(l <= r) {

    int mid = l + (r-l)/2;

    if(a[mid] == target) {
        // found
        break;
    }
    else if(a[mid] < target)
        l = mid + 1;
    else
        r = mid - 1;
}
```

29. BINARY SEARCH ON ANSWER

WHEN TO USE:

• Answer lies in a numeric range.

• Can write check(x) returning true/false.

• Feasibility is monotonic.

• "Minimum possible maximum", "maximum possible minimum", etc.

Minimum valid:

```
ll lo = low, hi = high;

while(lo < hi) {

    ll mid = lo + (hi-lo)/2;

    if(check(mid))
        hi = mid;
    else
        lo = mid + 1;
}
```

Maximum valid:

```
ll lo = low, hi = high;

while(lo < hi) {

    ll mid = lo + (hi-lo+1)/2;
```

```
    if(check(mid))
        lo = mid;
    else
        hi = mid - 1;
}
```

30. DSU / UNION FIND

WHEN TO USE:

• Dynamic connectivity.

• Merge components.

• Detect cycle in undirected graph.

• Kruskal MST.

• Connected component merging.

```
struct DSU {

    vector<int> parent, sz;

    DSU(int n) {
        parent.resize(n);
        sz.assign(n, 1);
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int x) {
        if(parent[x] == x)
            return x;

        return parent[x] = find(parent[x]);
    }

    bool unite(int a, int b) {

        a = find(a);
        b = find(b);

        if(a == b)
            return false;

        if(sz[a] < sz[b])
            swap(a, b);

        parent[b] = a;
        sz[a] += sz[b];

        return true;
    }
};
```

31. FENWICK TREE / BIT

WHEN TO USE:

• Point update + prefix/range sum.

• Count inversions.

• Frequency/order problems after coordinate compression.

• Simpler alternative to segment tree.

COMPLEXITY:

Update O(log n), query O(log n).

```
struct Fenwick {

    int n;
    vector<ll> bit;

    Fenwick(int n) : n(n), bit(n + 1, 0) {}

    void add(int idx, ll val) {
        for(; idx <= n; idx += idx & -idx)
            bit[idx] += val;
    }

    ll sum(int idx) {
```

```
        ll ans = 0;

        for(; idx > 0; idx -= idx & -idx)
            ans += bit[idx];

        return ans;
    }

    ll rangeSum(int l, int r) {
        return sum(r) - sum(l-1);
    }
};
```

32. SPARSE TABLE

WHEN TO USE:

• Static array.

• No updates.

• Many range min/max/GCD queries.

• Idempotent operations such as min/max/GCD.

COMPLEXITY:

Build O(n log n), query O(1) for min/max/GCD.

```
struct SparseTable {

    int n, LOG;
    vector<vector<int>> st;
    vector<int> lg;

    SparseTable(vector<int>& a) {

        n = a.size();
        LOG = 32 - __builtin_clz(n);

        st.assign(LOG, vector<int>(n));
        lg.resize(n + 1);

        for(int i = 2; i <= n; i++)
            lg[i] = lg[i/2] + 1;

        st[0] = a;

        for(int j = 1; j < LOG; j++) {
            for(int i = 0; i + (1 << j) <= n; i++) {
                st[j][i] =
                    min(st[j-1][i],
                        st[j-1][i + (1 << (j-1))]);
            }
        }
    }

    int query(int l, int r) {

        int j = lg[r-l+1];

        return min(st[j][l],
                   st[j][r-(1<<j)+1]);
    }
};
```

33. SEGMENT TREE

STATUS:

Already have your own template.

WHEN TO USE:

• Range query + updates.

• Sum/min/max/GCD/XOR.

• More flexible than Fenwick.

• Lazy propagation for range updates.

Keep your existing Segment Tree template here.

34. LAZY SEGMENT TREE

WHEN TO USE:

• Range update + range query.

• Example: add x to every element in [l,r], query range sum.

• More advanced version of segment tree.

Keep a separate tested template.

Do NOT improvise lazy propagation during a contest unless you are very comfortable with it.

35. ORDERED SET

WHEN TO USE:

• Maintain sorted unique values dynamically.

• Need lower_bound/upper_bound while inserting/deleting.

```
set<int> s;

s.insert(x);
s.erase(x);

auto it1 = s.lower_bound(x); // >= x
auto it2 = s.upper_bound(x); // > x
```

36. MULTISET

WHEN TO USE:

• Sorted collection allowing duplicates.

• Maintain current minimum/maximum dynamically.

• Sliding window multiset.

```
multiset<int> ms;

ms.insert(x);
ms.erase(ms.find(x));

int mn = *ms.begin();
int mx = *ms.rbegin();
```

37. PBDS ORDER STATISTICS TREE

WHEN TO USE:

• kth smallest dynamically.

• Number of elements smaller than x.

• Dynamic order statistics.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace __gnu_pbds;

template<class T>
using ordered_set =
    tree<T,
         null_type,
         less<T>,
         rb_tree_tag,
         tree_order_statistics_node_update>;
```

Usage:

```
ordered_set<int> s;
```

```
s.insert(x);

*s.find_by_order(k); // kth smallest, k is 0-indexed

s.order_of_key(x);   // number of elements < x
```

For duplicates, store pair<int,int>.

38. COORDINATE COMPRESSION

WHEN TO USE:

• Values are huge, e.g. 1e9/1e18.

• Only relative ordering matters.

• Need Fenwick/segment tree indexed by values.

```
vector<ll> v = a;

sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());

for(ll &x : a)
    x = lower_bound(v.begin(), v.end(), x) - v.begin();
```

39. GCD

WHEN TO USE:

• Greatest common divisor.

• Number theory.

• Simplifying fractions.

• Periodicity/divisibility.

```
ll g = gcd(a, b);
```

Manual:

```
ll gcdll(ll a, ll b) {
    while(b) {
        a %= b;
        swap(a, b);
    }
    return a;
}
```

40. LCM

WHEN TO USE:

• Least common multiple.

• Synchronization/period problems.

• Multiples/divisibility.

```
ll lcmll(ll a, ll b) {
    return a / gcd(a, b) * b;
}
```

Divide before multiply to reduce overflow risk.

41. EXTENDED EUCLIDEAN ALGORITHM

WHEN TO USE:

• Solve ax + by = gcd(a,b).

• Modular inverse when modulus is not necessarily prime.

• Linear Diophantine equations.

```
ll extgcd(ll a, ll b, ll &x, ll &y) {

    if(b == 0) {
        x = 1;
        y = 0;
        return a;
    }

    ll x1, y1;

    ll g = extgcd(b, a % b, x1, y1);

    x = y1;
    y = x1 - (a / b) * y1;

    return g;
}
```

42. FAST POWER / BINARY EXPONENTIATION

WHEN TO USE:

• a^b quickly.

• Modular exponentiation.

• Matrix exponentiation base operation.

• Any exponent up to huge values.

COMPLEXITY:

O(log b).

```
ll binpow(ll a, ll b, ll mod) {

    ll res = 1;

    while(b) {

        if(b & 1)
            res = res * a % mod;

        a = a * a % mod;
        b >>= 1;
    }

    return res;
}
```

43. MODULAR INVERSE

WHEN TO USE:

• Division under modulo.

• nCr modulo prime.

• Modular arithmetic.

If MOD is prime and gcd(a, MOD)=1:

```
ll inv = binpow(a, MOD - 2, MOD);
```

For non-prime modulus, use extended Euclid when inverse exists.

44. SIEVE OF ERATOSTHENES

WHEN TO USE:

• Need all primes <= N.

• Many prime queries.

• Prime checking for many numbers.

COMPLEXITY:

O(n log log n).

```
vector<bool> isPrime(n + 1, true);

isPrime[0] = isPrime[1] = false;

for(int i = 2; i * i <= n; i++) {

    if(isPrime[i]) {

        for(int j = i*i; j <= n; j += i)
            isPrime[j] = false;
    }
}
```

45. SMALLEST PRIME FACTOR / SPF

WHEN TO USE:

• Many prime factorizations.

• Need factorization of every number <= N.

• Divisor-related problems.

Build:

```
vector<int> spf(n + 1);

iota(spf.begin(), spf.end(), 0);

for(int i = 2; i * i <= n; i++) {

    if(spf[i] == i) {

        for(int j = i*i; j <= n; j += i) {

            if(spf[j] == j)
                spf[j] = i;
        }
    }
}
```

Factorize:

```
vector<pair<int,int>> factorize(int x) {

    vector<pair<int,int>> res;

    while(x > 1) {

        int p = spf[x];
        int cnt = 0;

        while(x % p == 0) {
            x /= p;
            cnt++;
        }

        res.push_back({p, cnt});
    }

    return res;
}
```

46. PRIME FACTORIZATION - SQRT(N)

WHEN TO USE:

• Only a few numbers need factorization.

• N can be large.

• No need to precompute SPF.

```
vector<ll> factorize(ll n) {

    vector<ll> factors;

    for(ll p = 2; p*p <= n; p++) {

        while(n % p == 0) {
```

```
            factors.push_back(p);
            n /= p;
        }
    }

    if(n > 1)
        factors.push_back(n);

    return factors;
}
```

47. NUMBER OF DIVISORS

WHEN TO USE:

• Count divisors of N.

• If prime factorization is known:

N = p1^a1 * p2^a2 * ...

answer = (a1+1)(a2+1)...

```
ll divisorCount(ll n) {

    ll ans = 1;

    for(ll p = 2; p*p <= n; p++) {

        if(n % p == 0) {

            int cnt = 0;

            while(n % p == 0) {
                n /= p;
                cnt++;
            }

            ans *= cnt + 1;
        }
    }

    if(n > 1)
        ans *= 2;

    return ans;
}
```

48. EULER TOTIENT PHI(N)

WHEN TO USE:

• Count integers in [1,n] coprime with n.

• Euler theorem.

• Multiplicative number theory.

```
ll phi(ll n) {

    ll ans = n;

    for(ll p = 2; p*p <= n; p++) {

        if(n % p == 0) {

            while(n % p == 0)
                n /= p;

            ans -= ans / p;
        }
    }

    if(n > 1)
        ans -= ans / n;

    return ans;
}
```

49. EULER PHI FOR ALL 1...N

WHEN TO USE:

• Need phi(i) for every i <= N.

```
vector<int> phi(n + 1);

for(int i = 0; i <= n; i++)
    phi[i] = i;

for(int p = 2; p <= n; p++) {

    if(phi[p] == p) {

        for(int j = p; j <= n; j += p)
            phi[j] -= phi[j] / p;
    }
}
```

50. nCr MODULO PRIME

WHEN TO USE:

• Many combinations.

• Fixed prime MOD.

• n up to manageable precomputation limit.

```
const ll MOD = 1e9 + 7;

vector<ll> fact(N + 1), invFact(N + 1);

fact[0] = 1;

for(int i = 1; i <= N; i++)
    fact[i] = fact[i-1] * i % MOD;

invFact[N] = binpow(fact[N], MOD - 2, MOD);

for(int i = N; i >= 1; i--)
    invFact[i-1] = invFact[i] * i % MOD;

ll C(int n, int r) {

    if(r < 0 || r > n)
        return 0;

    return fact[n]
         * invFact[r] % MOD
         * invFact[n-r] % MOD;
}
```

51. BIT MANIPULATION

WHEN TO USE:

• XOR/bitmask problems.

• Subset problems.

• Powers of two.

• Optimize states using bits.

Check bit:

```
if(x & (1LL << i))
```

Set:

```
x |= (1LL << i);
```

Clear:

```
x &= ~(1LL << i);
```

Toggle:

```
x ^= (1LL << i);
```

Lowest set bit:

```
x & -x
```

Remove lowest set bit:

```
x &= x - 1;
```

Count bits:

```
__builtin_popcount(x);
__builtin_popcountll(x);
```

52. SUBMASK ENUMERATION

WHEN TO USE:

• Bitmask DP.

• Enumerate every subset of a mask.

• Partition/subset problems.

```
for(int sub = mask; sub; sub = (sub-1) & mask) {
    // process sub
}
```

Including zero:

```
for(int sub = mask;; sub = (sub-1) & mask) {

    // process sub

    if(sub == 0)
        break;
}
```

53. MATRIX EXPONENTIATION

WHEN TO USE:

• Fibonacci.

• Linear recurrences.

• Need nth state of recurrence in O(log n).

• Transition matrix.

```
using Matrix = vector<vector<ll>>;

Matrix multiply(const Matrix& A, const Matrix& B) {

    int n = A.size();

    Matrix C(n, vector<ll>(n, 0));

    for(int i = 0; i < n; i++) {
        for(int k = 0; k < n; k++) {
            for(int j = 0; j < n; j++) {

                C[i][j] =
                    (C[i][j] + A[i][k] * B[k][j]) % MOD;
            }
        }
    }

    return C;
}

Matrix matpow(Matrix A, ll p) {

    int n = A.size();

    Matrix res(n, vector<ll>(n, 0));

    for(int i = 0; i < n; i++)
        res[i][i] = 1;

    while(p) {
```

```
        if(p & 1)
            res = multiply(res, A);

        A = multiply(A, A);
        p >>= 1;
    }

    return res;
}
```

54. INTERVAL MERGE

WHEN TO USE:

• Merge overlapping intervals.

• Scheduling.

• Range coverage.

```
sort(intervals.begin(), intervals.end());

vector<pair<int,int>> ans;

for(auto [l, r] : intervals) {

    if(ans.empty() || ans.back().second < l) {
        ans.push_back({l, r});
    }
    else {
        ans.back().second =
            max(ans.back().second, r);
    }
}
```

55. STL BINARY SEARCH

WHEN TO USE:

• Sorted container.

• Need first >= x or first > x.

```
auto it1 = lower_bound(a.begin(), a.end(), x);
// first element >= x

auto it2 = upper_bound(a.begin(), a.end(), x);
// first element > x
```

Count x:

```
int cnt =
    upper_bound(a.begin(), a.end(), x)
    - lower_bound(a.begin(), a.end(), x);
```

56. USEFUL STL

Sort:

```
sort(a.begin(), a.end());
sort(a.rbegin(), a.rend());
```

Remove duplicates:

```
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());
```

Maximum:

```
*max_element(a.begin(), a.end());
```

Minimum:

```
*min_element(a.begin(), a.end());
```

Sum:

```
accumulate(a.begin(), a.end(), 0LL);
```

Reverse:

```
reverse(a.begin(), a.end());
```

57. RANDOM

WHEN TO USE:

• Randomized algorithms.

• Random test generation.

• Avoid predictable rand().

```
mt19937 rng(
    chrono::steady_clock::now()
    .time_since_epoch().count()
);

int x = uniform_int_distribution<int>(l, r)(rng);
```

58. ADVANCED GRAPH ALGORITHMS

You already have your graph template, but for your long-term CP library, make sure it contains:

• BFS

• DFS

• Connected Components

• Bipartite Check

• Cycle Detection

• Topological Sort

• Kahn's Algorithm

• Dijkstra

• 0-1 BFS

• Bellman-Ford

• Floyd-Warshall

• DSU

• Kruskal

• Prim

• SCC (Kosaraju / Tarjan)

• Bridges

• Articulation Points

• LCA / Binary Lifting

• Euler Tour

• Tree Diameter

• Heavy-Light Decomposition

• Max Flow

• 2-SAT

59. ADVANCED DATA STRUCTURES

For higher Codeforces ratings, eventually add:

• Lazy Segment Tree
• Fenwick Tree variants
• Sparse Table
• DSU on Tree
• Heavy-Light Decomposition
• Persistent Segment Tree
• Treap
• Ordered Set / PBDS
• Binary Trie
• Sqrt Decomposition
• Mo's Algorithm
• Li Chao Tree

60. ADVANCED STRING ALGORITHMS

Eventually add:

• Rolling Hash
• Double Hash
• Manacher
• KMP
• Z Algorithm
• Trie
• Aho-Corasick
• Suffix Array
• LCP Array
• Suffix Automaton

61. ADVANCED NUMBER THEORY

Eventually add:

• Chinese Remainder Theorem
• Linear Diophantine Equation
• Mobius Function
• Divisor Sieve
• Prime Factor Sieve
• Modular Inverse
• Fermat's Little Theorem
• Euler's Theorem
• Lucas Theorem
• Fast Doubling Fibonacci
• Miller-Rabin
• Pollard Rho

62. IMPORTANT PATTERN: PREFIX + SUFFIX ANSWER

WHEN TO USE:

When the problem asks you to split an array/string at every position and combine the best result on the left and right.

Pattern:

```
vector<ll> prefAns(n);
vector<ll> suffAns(n);

for(int i = 0; i < n; i++) {
    // calculate best answer for [0..i]
}

for(int i = n-1; i >= 0; i--) {
    // calculate best answer for [i..n-1]
}

for(int i = 0; i + 1 < n; i++) {
    // combine prefAns[i] and suffAns[i+1]
}
```

63. HOW TO CHOOSE THE ALGORITHM

If you see:

"Many range sums"

-> Prefix Sum.

"Many range updates, final array only"

-> Difference Array.

"Longest/shortest valid subarray"

-> Sliding Window / Two Pointers.

"Sorted array + pair condition"

-> Two Pointers / Binary Search.

"Next greater/smaller"

-> Monotonic Stack.

"Window maximum/minimum"

-> Monotonic Queue.

"Linked list middle/cycle"

-> Fast + Slow Pointer.

"Prefix of words"

-> Trie.

"Pattern matching"

-> KMP / Z.

"Maximum XOR"

-> Binary Trie.

"Dynamic connectivity"

-> DSU.

"Point update + range sum"

-> Fenwick Tree.

"Range query + update, complex operation"

-> Segment Tree.

"Static range min/max/GCD"

-> Sparse Table.

"Dynamic kth smallest"

-> PBDS.

"Huge values but ordering matters"

-> Coordinate Compression.

"Minimum/maximum answer with monotonic feasibility"

-> Binary Search on Answer.

"Many prime queries"

-> Sieve.

"Many factorisations <= N"

-> SPF.

"Only one/few large factorisations"

-> Trial division sqrt(N).

"Need a^b"

-> Binary Exponentiation.

"Combinations modulo prime"

-> Factorial + Inverse Factorial.

"XOR/subset/state compression"

-> Bitmask.

"nth Fibonacci/linear recurrence"

-> Matrix Exponentiation / Fast Doubling.

64. COMPLEXITY CHEAT SHEET

O(1)

• Prefix sum query

• Sparse table query

• Basic math operations

O(log n)

• Binary search

• Fenwick update/query

• Segment tree query/update

• DSU amortized almost O(1)

• PBDS operations

• Fast power

O(n)

• Prefix/suffix arrays

• Sliding window

• Two pointers

• Kadane

• Monotonic stack

• Monotonic queue

• KMP

• Z algorithm

• Manacher

• Sieve is slightly above linear

O(n log n)

• Sorting

• Sparse table build

• Coordinate compression

• Many tree algorithms

O(n log^2 n) / higher

• Some HLD / advanced structures

O(2^n)

• Basic subset enumeration

O(3^n)

• Some subset-partition DP using submask enumeration

65. FINAL REVISION PRIORITY

MUST KNOW COLD:

1. Prefix/Suffix

2. Difference Array

3. Sliding Window

4. Two Pointers

5. Binary Search

6. Binary Search on Answer

7. Stack/Queue/Deque

8. Monotonic Stack

9. Monotonic Queue

10. Kadane

11. DSU

12. Fenwick

13. GCD/LCM

14. Fast Power

15. Sieve

16. Factorisation

17. SPF

18. Bit Manipulation

19. Coordinate Compression

20. Basic Trie

21. KMP/Z

SHOULD KNOW:

22. Sparse Table

23. PBDS

24. Binary Trie

25. nCr

26. Euler Phi

27. Extended GCD

28. Matrix Exponentiation

29. Manacher

30. Rolling Hash

ADVANCED:

31. Lazy Segment Tree

32. Mo's Algorithm

33. HLD

34. DSU on Tree

35. Persistent Segment Tree

36. SCC

37. Bridges/Articulation

38. LCA

39. Max Flow

40. 2-SAT

41. Aho-Corasick

42. Suffix Array

43. Suffix Automaton

44. Miller-Rabin

45. Pollard Rho

46. FFT/NTT

47. Li Chao Tree

48. Treap

IMPORTANT:

Do not memorize templates blindly.

For every template, remember:

• What problem pattern triggers it?

• What invariant does it maintain?

• What is the complexity?

• What are the indexing conventions?

• What edge cases break it?

A template is useful only when you can recognize WHEN to use it.